Hashing Schemes

This paper will investigate the conjecture below.
For reasonable values of the load factor α, open addressing with linear probing provides better performance than chaining.

Introduction

Hash tables have many wide-ranging uses across the modern technological landscape. From being a crucial part of key-value database systems like *Redis* and *Cassandra* to being fundamental components to compiler design and operating systems, Hash tables are foundational [1].

Hash tables come in various forms, each with different choices made about collision handling, resizing, and structure each decision having benefits, downsides and use cases. This paper will focus on comparing the choice of collision resolution between open addressing with linear probing and chaining in terms of performance. For this paper, performance will be defined by an operation cost metric counting the number of probes, *definition of probes will be provided in the methodology section*.

Load Factor
Both linear probing and chaining use a load factor value denoted α and defined as $\alpha = \frac{N}{M}$ where N and M correspond to the number of inserted key value pairs, and the number of table slots respectively.

The open addressing with linear probing approach sets the maximum size of a table slot as one, upon collision the program checks the next table slot from the current index, looping back around at the last index. Thus for this case the load factor α can be interpreted as the percentage of the table that is full, never exceeding 1.

For the chaining approach, because the table slots hold a linked list structure for which a collision will attach the key value pair to the end of the list, α will thus represent the expected number of nodes in each linked-list represented by a table slot for a hash function that follows the uniform hashing assumption[2].

Observing the behavior of α for various values will allow use to monitor the performance of both. This paper will investigate and summarize the following conjecture:
For reasonable values of the load factor α, open addressing with linear probing provides better performance than chaining.

It will do so by exploring what we should theoretically expect the conclusion to be. Then we will explore implemented experiments of hash maps to see how the outcome of that compares to

the predicted conclusion.

Methodology

This study will evaluate open addressing with linear probing and chaining using two approaches: (i) theoretical expectations for probe counts under the uniform hashing assumption, and (ii) empirical measurements from baseline C++ implementations.

Performance metric.
Performance is primarily measured by *probe count*, defined as the number of memory locations inspected during an operation. For linear probing, a probe is counted each time a table slot is read and checked while searching for a key or insertion position. Similarly, for chaining, will be the initial table access (one bucket inspection) plus one additional probe per linked-list node visited during traversal. We will report probe counts for the operations of insert and find (successful and unsuccessful).

Rationale for probe-based measurement.
Probe counts provide a measurement of work performed that is not effected by hardware or platform, whereas wall-clock timing and CPU cycle counts can vary substantially with compiler optimizations, caching effects, background processes and runtime environments. Timing measurements are thus collected only as supplementary evidence and are not treated as the primary basis for conclusions.

Implementation and environment.
Both the hash table variants are implemented in C++20 and compiled using GNU g++. Experiments are executed on an AMD Ryzen 7 7840HS processor with 16GB of installed physical memory (RAM) running Ubuntu on Windows 11. All experiments use a fixed random seed for reproducibility.

The creation of the test files that the hash table variants use was done using python 3, primarily due to ease of use. More about the test files is explained in the results section.

Experimental setup
The experiment will only use 64-bit unsigned integers as keys and values, hashed using the `std::hash<uint64_t>` provided by the GNU libstdc++ standard library and reduced to the table size M using the Modulo operator. In our analysis, we treat this hash function as an approximation to the Uniform Hashing Assumption, thus any systematic deviation from uniformity is a potential source of error.

Table size and load factor
For each target load factor α, we fix a table size M and insert

$$N = \lfloor \alpha \cdot M \rfloor$$

distinct keys. Resizing is disabled: both hash tables are created once at size M and never reallocated. This intentionally isolates collision resolution behavior as α changes. M was chosen to be constant for the experiment at 200,000 in order to balance efficiency and quantity during testing.

Test Generation
For each α, a Python program (`make_test.py`) generates a test file with a seeded random number generator (RNG) containing a sequence of operations:

1. Build phase: N inserts of unique keys.
2. Successful lookup phase: Find operations where keys are sampled from the inserted set in order to guarantee hits.
3. Unsuccessful lookup phase: Find operations where keys are sampled from a disjoint set in order to guarantee misses.

The amount of both types of lookups will be constant throughout the experiment and equal to M.

Test Reporting
The test file produced by (`make_test.py`) is parsed through a C++20 runner program (`runner.cpp`), which takes the test file, and applies each operation to both hash tables to ensure identical input ordering. A sample test file is provided below:

```
20 (defines M for both tables)
I 5 4 (Insert with key = 5 and value = 4)
S 5 (Search for key 5)
R (Prints out probe count up to that point)
```

To obtain per-operation averages without logging every step, the test files contain report markers (R) placed at the boundaries of each test phase. The runner prints cumulative probe totals for each structure at each marker in the test file, per phase average probe cost is computed by taking the difference of successive totals and dividing out the number of operations in that phase

Table
The table consists of the cumulative probe totals at the phase report markers (T1 = after inserts, T2 = after successful finds, T3 = after unsuccessful finds) as well as the following for columns:

- α (Load factor)
- N (number of inserted keys): $N = \lfloor \alpha \cdot M \rfloor$
- LP_{succ} (avg probes per successful Find in linear probing)

- CH_{succ} (avg probes per successful Find in chaining)
- LP_{fail} (avg probes per unsuccessful Find in linear probing)
- CH_{fail} (avg probes per unsuccessful Find in chaining)

Each of the average probes is calculated by taking the delta of the probe totals surrounding its stage and dividing it over the amount of each operation (which was 200,000 for each action for consistency).

Theoretical Prediction

Linear Probing approximation

In this paper, I use the standard asymptotic analysis of linear probing given by Knuth [3, Section 6.4].to obtain closed-form approximations for the expected number of probes for successful and unsuccessful searches. Under the uniform hashing assumption and load factor $\alpha = \frac{N}{M}$ < 1, linear probing creates contiguous runs of occupied table slots (primary clusters). The cost of an unsuccessful search is dependent on the length of the cluster encountered since it must iterate through the table sequentially until it reaches the first empty cell. Knuth carries out the full probabilistic analysis and obtains:

$$E\left[\text{unsuccessful probes}\right] \approx \frac{1}{2}\left(1 + \left(\frac{1}{1-\alpha}\right)^2\right)$$

A successful search follows the same probe sequence created at insertion time, so its expected cost can be derived by averaging insertion cost over the build-up of the table, this yields:

$$E\left[\text{successful probes}\right] \approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$$

I will use Knuth's approximation results as theoretical predictions and validate them empirically.

Unsuccessful chaining approximation

To apply the memory-location inspected approach to probe count to probing we will consider the act of checking the table slot to access the linked list structure as a necessary probe, as well as each examined node of the structure to be another probe count.

Consider a randomly selected sequence has been inserted into a table, $k_1, \ldots, k_N$ in U by a hash function h(k) that satisfies the uniform hashing assumption.

Define a random variable X_{ij} as follows.

$$X_{ij} = \begin{cases} 1 & \text{if } h(k_i) = j \\ 0 & \text{otherwise} \end{cases}$$

Since the possible outputs of h(k) % M are 0, 1 , ... , M-1, and we are following the uniform hashing assumption then we can assume:

$$P(X_{ij} = 1) = \frac{1}{M}$$

And thus

$$E[X_{ij}] = P(X_{ij} = 1) \cdot 1 + P(X_{ij} = 0) \cdot 0 = P(X_{ij} = 1) = \frac{1}{M}$$

Suppose the length of a chain to be:

$$L_j = \sum_{i=1}^{N} X_{ij}$$

If we solve for the expected value of L_j we have that:

$$E[L_j] = E\left[\sum_{i=1}^{N} X_{ij}\right]$$

Applying linearity of expectation:

$$E\left[\sum_{i=1}^{N} X_{ij}\right] = \sum_{i=1}^{N} E[X_{ij}] = \sum_{i=1}^{N} \frac{1}{M} = \frac{N}{M} = \alpha$$

So the expected length of a chain of a specific bucket j, is equal to α, which for an unsuccessful search is equivalent to the expected number of nodes visited. Adding one probe to the number of nodes visited to account for the initial bucket check, we have that the amount of possible probes in a bucket (an unsuccessful search) is:

$$\alpha + 1$$

Successful chaining approximation

Following our definition of a probe, a successful search for a key k_i in a chained hash table will require:

- 1 Probe to access the initial bucket
- 1 Probe per node for every key k_j that was inserted before k_i and hashed to the same slot
- 1 Probe to examine the node containing k_i

To determine the closed form probe count approximation for a successful search in a chaining implementation, we will again refer to a derivation from Knuth [3, Section 6.4].

Knuth measures the expected number of key comparisons (or nodes examined). He proves that for a hash table with separate chaining, the average number of nodes examined during a successful search is:

$$1 + \frac{\alpha}{2} - \frac{\alpha}{2N}$$

Because our memory-location inspected approach requires an additional initial probe to check the table slot itself, our expected successful probe count simply adds 1 to Knuth's baseline, resulting in:

$$2 + \frac{\alpha}{2} - \frac{\alpha}{2N}$$

Which for large values values of N (around 70), approaches:

$$2 + \frac{\alpha}{2}$$

Data and Hypotheses

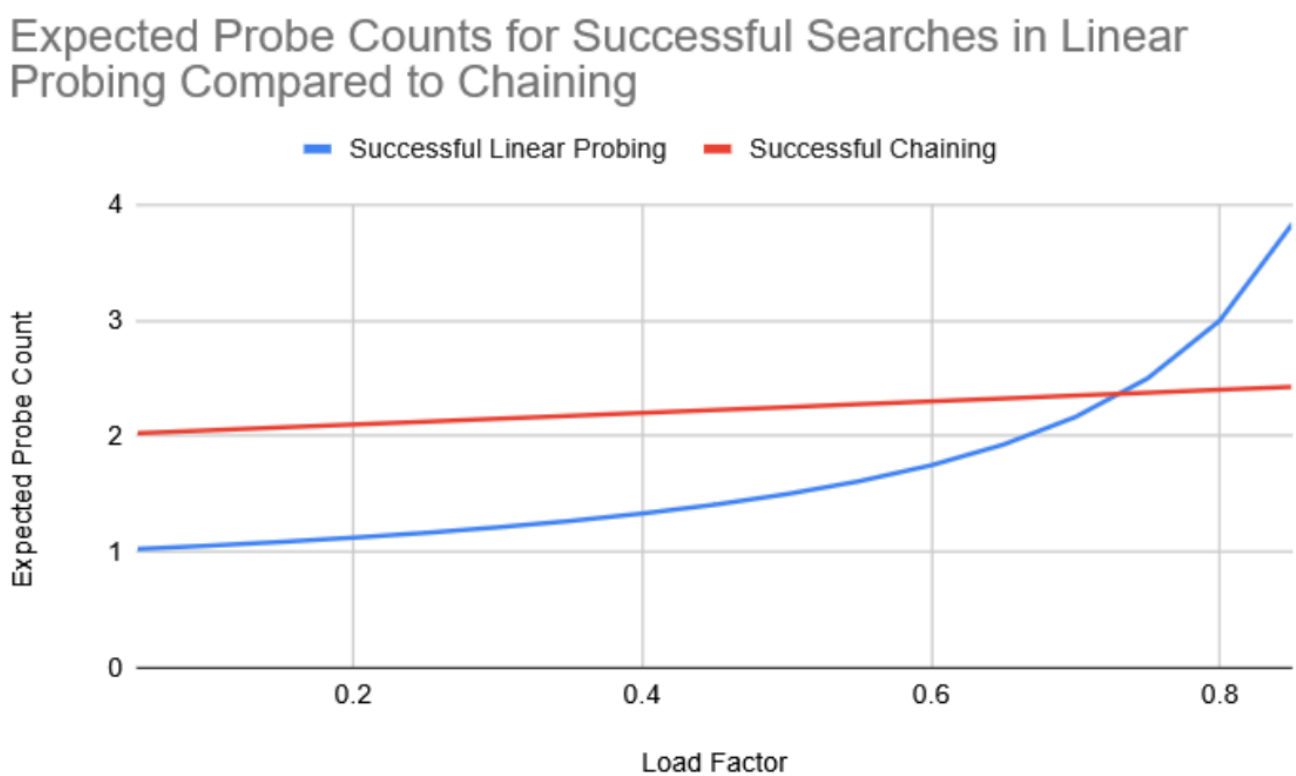

Figure 1: Expected Probe Counts for Successful Searches in Linear Probing Compared to Chaining

Figure 1 plots the expected probe count for successful searches as a function of load factor α for both linear probing and chaining. The domain α is restricted to $\alpha <$ 0.85 for clarity since our closed form approximations for linear probing diverge as $\alpha \to 1$ which disproportionately scales the y-axis. We can see that at low to moderate load factors, linear probing requires fewer probes than chaining, we can find the exact value by finding the intersection:

$$\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right) = 2 + \frac{\alpha}{2}$$

$$\alpha = \sqrt{3} - 1 \approx 0.732$$

Thus we can propose the following hypotheses

[H.1] For values of α such that $0 < \alpha < 0.732$, successful searches with linear probing, by our definitions, theoretically provides better performance than chaining.

[H.2] For values of α such that $0.732 < \alpha < 1$, successful searches with chaining, by our definitions, theoretically provides better performance than linear probing.

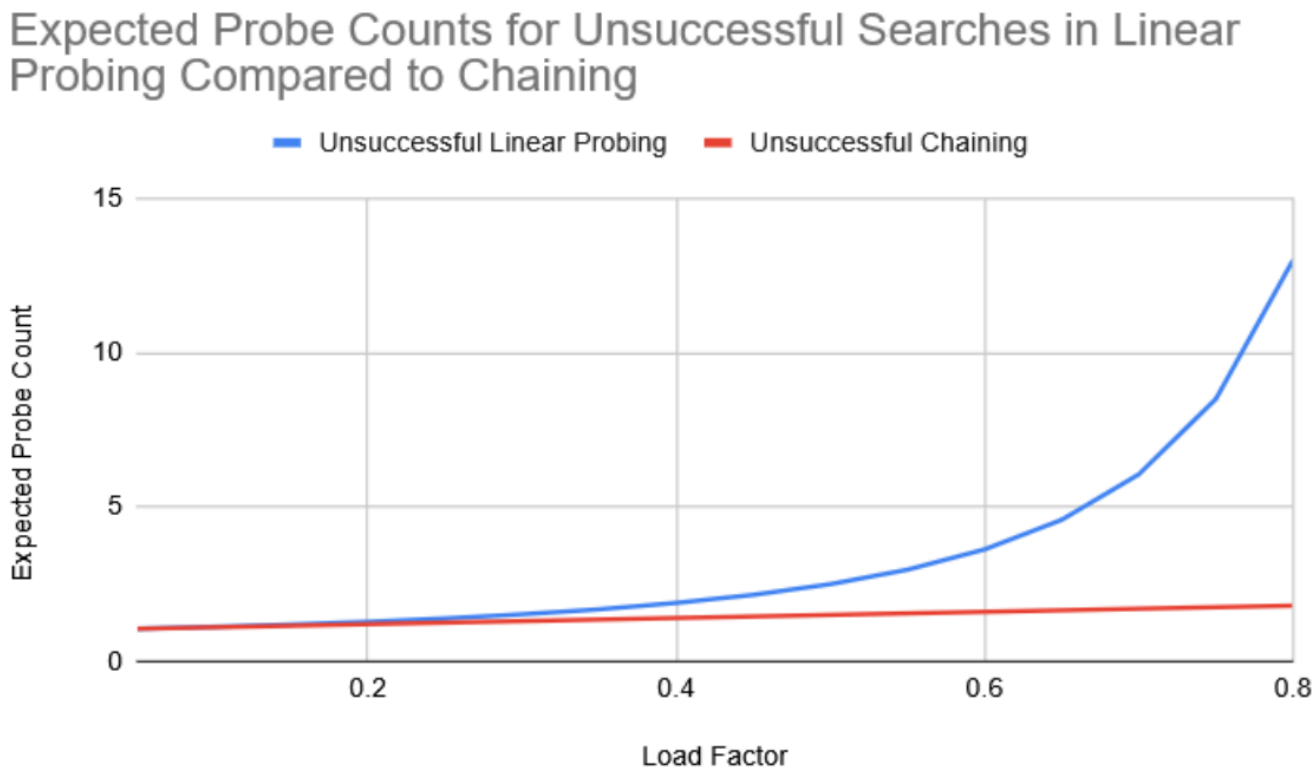

Figure 2: Expected Probe Counts for Unsuccessful Searches in Linear Probing Compared to Chaining

Figure 2 plots the expected probe count for unsuccessful searches as a function of load factor α for linear probing and chaining. For similar reasons as for Figure 1, we only show α up to 0.8. We can see that the expected probe count for chaining is always less than the expected probe count for linear probing. This allows us to make the following hypotheses:

[H.3] For values of α such that $0 < \alpha < 1$, unsuccessful searches with chaining, by our definitions, theoretically provide better performance than linear probing.

Results

After running the tests for the set of α (0.05,0.10,…,0.90), with $N = \lfloor \alpha \cdot M \rfloor$,

alpha	N	T1 LP	T1 CH	T2 LP	T2 CH	T3 LP	T3 CH	LP succ	CH succ	LP fail	CH fail
0.05	10000	10303	10288	216407	416081	427362	626186	1.03052	2.028965	1.054775	1.050525
0.1	20000	21121	21025	232469	431419	455781	651429	1.05674	2.05197	1.11656	1.10005
0.15	30000	32624	32253	250085	447225	488424	677221	1.087305	2.07486	1.191695	1.14998
0.2	40000	44962	44000	269819	464016	525772	703971	1.124285	2.10008	1.279765	1.199775
0.25	50000	58252	56205	291402	481160	568870	731373	1.16575	2.124775	1.38734	1.251065
0.3	60000	72686	68917	314862	498623	618797	758926	1.21088	2.14853	1.519675	1.301515
0.35	70000	88684	82126	342655	517039	678305	787220	1.269855	2.174565	1.67825	1.350905
0.4	80000	106770	95905	373436	535451	751531	815640	1.33333	2.19773	1.890475	1.400945
0.45	90000	127198	110216	409897	555095	841207	845183	1.413495	2.224395	2.15655	1.45044
0.5	100000	150427	125008	451811	575524	952967	875535	1.50692	2.25258	2.50578	1.500055
0.6	120000	210924	155954	561613	615716	1288349	935320	1.753445	2.29881	3.63368	1.59802
0.7	140000	305600	189032	741830	658885	1963668	998281	2.18115	2.349265	6.10919	1.69698
0.8	160000	487902	223925	1095250	703697	3766448	1063441	3.03674	2.39886	13.35599	1.79872
0.9	180000	1010147	260990	2118876	750471	12512058	1130172	5.543645	2.447405	51.96591	1.898505

Figure 3: Resulting table after running tests

Testing [H.1] and [H.2]

Across low to moderate load factors, linear probing achieves a lower average probe cost than chaining for successful lookups. For example, at α=0.05, LP_{succ} ≈ 1.031 while CH_{succ} ≈ 2.029. This separation persists until the crossover point. As predicted by the intersection of our theoretical models at α ≈ 0.732, the empirical crossover occurs between α = 0.70 and α = 0.80. At α = 0.80, LP_{succ} (3.037) exceeds CH_{succ} (2.399), which provides strong validation for both [H.1] and [H.2].

Testing [H.3]

For unsuccessful lookups, chaining dominates linear probing across the entire tested range. While both methods begin near 1.0 probes at α=0.05, linear probing's costs grow exponentially as primary clusters form, reaching 51.966 probes at α=0.90. In contrast, the chaining implementation grows linearly, tracking the α+1 model closely (for example at α= 0.90, CH_{fail} ≈1.90). These results provide conclusive support for [H.3], demonstrating that chaining is vastly superior for misses at high load factors.

Conclusions and limitations

Conclusion

This paper investigated the conjecture that LP provides better performance than CH for reasonable values of the load factor α. Based on the combination of theoretical analysis and empirical data, the final verdict is that the conjecture is partially supported but depends heavily on the operation type and the specific load factor.

The investigation revealed a clear divide in performance based on if a search is successful or unsuccessful:

Successful Searches: The evidence supports the conjecture for low to moderate load factors. Theoretically and empirically, the linear probing implementation is more efficient than the

chaining one until α≈0.732. The data showed the crossover occurs between 0.70 and 0.80, where linear probing's probe count (3.037) finally exceeds chaining's (2.399).

Unsuccessful Searches: The evidence refutes the conjecture. The chaining implementation consistently outperformed LP across every tested load factor. While chaining stays very efficient (under 2 probes even at α=0.90), linear probing's performance degrades exponentially, reaching an average of nearly 52 probes for a single miss at high loads.

Limitations
Several limitations must be acknowledged to contextualize the results.

1. Simplification of Data types: This study exclusively utilized 64-bit unsigned integers for both keys and values. In practical applications involving complex objects or variable-length strings, that may slow down and drastically change the the performance
2. Static Hash Table: Resizing was intentionally ignored to focus on the impact of the load factor on collision resolution. However in application settings, hash tables typically resize dynamically, which could cause linear probing to slow down depending on the rules set for resizing.
3. Exclusion of Deletions: Since deletion performance in large part is similar to that of a search, we have chosen to ignore it for this paper since there are numerous different optimizations to be made depending on tombstone resetting and clearing. This would have added complexities that the paper is not focused on.
4. Ignoring cache locality: Since we have focused our performance on probe count and not the speed of memory access, we are ignoring cache locality. Different systems have different implementations of CPU cache structure and thus the linked list in the chained implementation could have somewhat drastically different performance based on the system.

Ultimately, while linear probing is faster for successful lookups in tables that are half full, chaining is the more robust choice for general-purpose use where misses are frequent or the table is nearly full.

Bibliography

[1] P. A. Gagniuc and M. Togan, "Hash Tables as Engines of Randomness at the Limits of Computation: A Unified Review of Algorithms," Algorithms, vol. 18, no. 12, Art. no. 804, 2025, doi: 10.3390/a18120804.

[2] B. Bird, "Hashing I," CSC 226: Algorithm Science, Univ. of Victoria, Lecture 20 slides, Jan. 2026. Online. Available: Brightspace.

[3] D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.